Distributed Mutual Exclusion Explorer

Interactive Tools for Learning Distributed Systems Concepts (Variant 3)

Lei Ding

Student ID: K21029011

Programme: BSc Computer Science

Supervisor: Angelos Georgoulas

Department of Informatics, King's College London

March 13, 2026

Originality Avowal

This report is submitted as part of the requirements for the Final Year Individual Project. It is the result of the author's own work except where explicitly indicated in the text. All sources are acknowledged by appropriate references. This report has not been submitted for any other assessment at this or any other institution.

Abstract

Distributed mutual exclusion is a foundational distributed systems problem, yet its behaviour is difficult to learn from static descriptions because correctness depends on concurrency, ordering, and failure assumptions. This project delivers a browser-based interactive teaching tool that visualises distributed mutual exclusion using two representative approaches: a token-based Token Ring model and a teaching-oriented Ricart–Agrawala (RA) prototype. The tool supports step-by-step execution, scripted demonstrations, fault injection (token loss, crash/recover, and message loss in RA), and evidence export (state JSON, trace TXT, preview PNG) for reproducible evaluation. Correctness is evaluated primarily via the safety property (mutual exclusion), while fault scenarios are used to illustrate the separation between safety and liveness. Testing combines a manual evidence-driven test plan with dependency-free automated smoke tests executed in continuous integration. The resulting artefact provides an accessible environment for exploring distributed coordination and for producing traceable evaluation evidence.

Acknowledgements

The author would like to thank the project supervisor for guidance and feedback throughout the project.

Contents

Originality Avowal **i**

Abstract **ii**

Acknowledgements **iii**

1 Introduction **1**

1.1 Motivation 1

1.2 Project Aim and Objectives 1

1.3 Scope 2

1.4 Contributions 2

1.5 Report Structure 2

2 Background and Related Work **3**

2.1 Distributed Mutual Exclusion 3

2.2 Logical Time and Ordering 3

2.3 Failures and Assumptions 3

2.4 Algorithm Visualisation and Learning 4

2.5 Cognitive Load and Multimedia Learning 4

2.6 Positioning of This Project 4

3 Specification and Design **5**

3.1 Requirements Approach 5

3.2 Functional Requirements 5

3.3 Non-functional Requirements 5

3.4 High-level Architecture 6

3.5 Execution Model: Step Semantics 6

3.6 Fault Injection Design 6

3.7 Evidence and Traceability Design 7

3.8 UI Design Rationale 7

4 Specification and Design **8**
4.1 Requirements Approach . . . 8
4.2 Functional Requirements . . . 8
4.3 Non-functional Requirements . . . 8
4.4 High-level Architecture . . . 9
4.5 Execution Model: Step Semantics . . . 9
4.6 Fault Injection Design . . . 9
4.7 Evidence and Traceability Design . . . 10
4.8 UI Design Rationale . . . 10

5 Testing **11**
5.1 Testing Objectives . . . 11
5.2 Manual Evidence-driven Testing . . . 11
5.3 Automated Smoke Tests . . . 11
5.4 Continuous Integration . . . 11
5.5 Limitations . . . 12

6 Evaluation **13**
6.1 Evaluation Criteria . . . 13
6.2 Correctness: Safety Invariant . . . 13
6.3 Fault Scenarios and Teaching Value . . . 13
6.3.1 Token Loss and Recovery . . . 13
6.3.2 Crash and Recover . . . 13
6.3.3 RA Tie-break under Equal Timestamps . . . 13
6.3.4 RA Message Loss: Safety vs Liveness . . . 15
6.4 Usability Evaluation . . . 15
6.5 Reproducibility . . . 15
6.6 Threats to Validity . . . 17

7 Professional Issues **18**
7.1 Accessibility . . . 18
7.2 Privacy and Data Protection . . . 18
7.3 Sustainability and Maintainability . . . 18
7.4 Reproducibility and Research Integrity . . . 18
7.5 Ethics and Responsible Communication . . . 18
7.6 Standards and Evaluation Methodology . . . 19

8 Conclusion and Future Work **20**

8.1 Conclusion 20

8.2 Future Work 20

A User Manual **23**

B Test Cases **24**

List of Figures

6.1 Token Ring: token loss halts progress until regeneration. 14
6.2 Token Ring: crash and recover demonstration. 14
6.3 RA: deterministic tie-break (same timestamp) using numeric PID ordering. . . . 15
6.4 RA: dropping an in-flight message can cause a stall; the trace explains which REPLY is missing. 16
6.5 RA: drop-next-send can cause a stall while safety remains OK. 16

List of Tables

Chapter 1

Introduction

1.1 Motivation

Distributed systems underpin modern computing, but their behaviour is often difficult to understand from static material because correctness depends on concurrency, message ordering, and assumptions about failures [25, 5, 13]. Mutual exclusion is a classic example: a set of distributed processes must coordinate access to a shared critical section, without relying on shared memory or a global clock.

In practice, learners commonly struggle to connect algorithm descriptions to concrete executions. The challenge is amplified when faults are introduced: safety properties must be preserved, yet progress (liveness) may be lost under weaker assumptions. This motivates interactive tooling that enables controlled experimentation and observation.

1.2 Project Aim and Objectives

The aim of this project is to create an interactive educational tool that visualises distributed mutual exclusion and supports exploration under faults and recovery.

The objectives are:

- Implement a technically correct simulation that preserves the safety invariant of mutual exclusion.
- Provide step-by-step execution with an explanatory trace to make causality and dependencies visible.
- Support representative algorithm families for comparison (token-based and message-based).
- Support fault injection and recovery mechanisms to illustrate safety vs liveness trade-offs.
- Provide reproducible evaluation evidence via exports (state JSON, trace TXT, preview PNG) and versioned releases.
- Evaluate correctness, usability, and reproducibility using a structured test plan and heuristic analysis.

1.3 Scope

Two mutual exclusion mechanisms are implemented:

- **Token Ring** (token-based), which provides an intuitive mechanism for exclusivity via token ownership.
- **Ricart–Agrawala (RA)** (message-based) [19], presented as a teaching-oriented prototype using Lamport-style logical time [12].

The tool focuses on clarity and reproducibility rather than high-fidelity network timing. The RA component intentionally demonstrates that liveness may fail under message loss, while safety remains preserved; retransmission and failure detectors are not implemented.

1.4 Contributions

The main contributions are:

- A dependency-free browser-based simulator with interactive stepping and scripted demos.
- Fault injection controls (token loss, crash/recover, RA message drops) and recovery mechanisms.
- Trace explanations, including explicit stall reasons (e.g., missing REPLY dependencies).
- A reproducible evidence pipeline (JSON + trace + PNG) with indices that link report figures to raw artefacts.
- Automated smoke tests and continuous integration to reduce regression risk.

1.5 Report Structure

Chapter 2 reviews mutual exclusion algorithms and relevant learning literature. Chapter 4 specifies requirements and design decisions. Chapters **??**–6 describe implementation, testing, and evaluation. Chapter 7 discusses professional issues such as accessibility and reproducibility. Chapter 8 concludes and outlines future work.

Chapter 2

Background and Related Work

2.1 Distributed Mutual Exclusion

Distributed mutual exclusion requires coordinating access to a critical section across processes that communicate by message passing. Two key correctness dimensions are commonly distinguished:

- **Safety:** at most one process may be in the critical section at any time.
- **Liveness:** requesting processes eventually enter the critical section under suitable assumptions.

This safety–liveness separation is fundamental in distributed systems [13, 1, 26].

Mutual exclusion algorithms can be categorised by their coordination mechanism:

- **Message-based algorithms:** e.g., Ricart–Agrawala [19] and related quorum approaches such as Maekawa's algorithm [14].
- **Token-based algorithms:** e.g., Suzuki–Kasami [23] and ring-based token circulation approaches.
- **Tree-based variants:** e.g., Raymond's algorithm [18].

This project implements a Token Ring model to illustrate token-based exclusivity, and a teaching-oriented RA prototype to illustrate ordering, replies, and the impact of message loss.

2.2 Logical Time and Ordering

Many message-based coordination algorithms rely on logical ordering rather than real-time ordering. Lamport introduced logical clocks to define a happens-before relation and impose a consistent order on distributed events [12]. In RA, logical timestamps are used to prioritise requests; when timestamps tie, an additional deterministic rule (such as process ID) is required.

2.3 Failures and Assumptions

Distributed algorithms depend on system assumptions. Classic impossibility results such as FLP highlight that consensus-like agreement properties cannot be guaranteed under certain asyn-

chronous failure models [6]. For mutual exclusion, message loss and crashes can break liveness even when safety remains preserved, unless additional mechanisms (timeouts, retransmissions, failure detectors) are introduced [4].

This project uses fault injection to make such dependencies observable. In RA, message loss is used to demonstrate liveness failure in a controlled manner.

2.4 Algorithm Visualisation and Learning

Algorithm visualisation has long been studied as a means to improve engagement and comprehension. Reviews and meta-analyses indicate that effectiveness depends on interactivity and learner engagement rather than animation alone [16, 8]. Educational visualisation research also notes the need for careful design to prevent misconceptions [21, 22, 27].

The tool implemented in this project emphasises:

- interactivity (step-by-step execution and fault injection),
- explicit explanations (trace and stall reasons),
- and reproducibility (scripted demos and exported evidence).

2.5 Cognitive Load and Multimedia Learning

Learning complex distributed behaviours can impose high intrinsic cognitive load. Cognitive load theory argues that instructional design should minimise extraneous load and support germane processing [24]. Multimedia learning principles emphasise aligning representations with learner processing capabilities [15]. Minimally guided discovery can be ineffective for novices without scaffolding [11]. These perspectives motivate the design choice of a discrete step engine and concise trace explanations to scaffold understanding.

2.6 Positioning of This Project

The artefact aligns with the educational goals of algorithm visualisation: it supports active exploration, makes hidden dependencies visible, and provides reproducible scenarios for teaching and reporting. Compared to generic simulators, the focus is deliberately narrow: distributed mutual exclusion, faults, and explanation.

Chapter 3

Specification and Design

3.1 Requirements Approach

A lightweight specification was derived using a structured requirements mindset consistent with software requirements guidance [9]. The focus is on clear functional behaviour, non-functional constraints, and evidence-driven evaluation.

3.2 Functional Requirements

The core functional requirements are:

- **FR1: Configuration.** Users can select an algorithm (Token Ring or RA) and configure the number of processes.
- **FR2: Interactive execution.** Users can issue requests, release the critical section, and progress the system via Step/Run/Pause/Reset.
- **FR3: Visual status.** The UI must clearly show process states, the critical section owner, and (for RA) messages in flight.
- **FR4: Fault injection and recovery.** Users can inject representative faults and observe effects; recoveries must be supported where appropriate.
- **FR5: Scripted demos.** Repeatable scenarios can be loaded for teaching and evidence capture.
- **FR6: Evidence export.** The system exports a state snapshot and a trace, plus a visual PNG preview, enabling reproducible evidence.

3.3 Non-functional Requirements

The main non-functional requirements are:

- **NFR1: Safety.** The mutual exclusion safety property must be preserved.
- **NFR2: Clarity.** Learners must be able to explain why progress occurs or stalls (trace explanations, explicit stall reasons).

- **NFR3: Reproducibility.** Scripted scenarios and exports must allow re-running and verifying results.
- **NFR4: Portability.** The tool must run in a modern browser without external dependencies.
- **NFR5: Maintainability.** The codebase should remain small and modular to enable iteration and testing.

3.4 High-level Architecture

The design follows separation of concerns [7]:

- **Core simulation module:** maintains model state and defines algorithm step semantics.
- **UI controller:** renders tables, trace, and the preview canvas; wires user actions to core operations.

This enables headless automated tests against the core logic while keeping the UI implementation simple.

3.5 Execution Model: Step Semantics

A key design decision is discrete stepping. Each Step performs one unit of progress:

- **Token Ring:** token passing or CS entry (if the holder is requesting).
- **RA:** delivery of exactly one queued message (REQUEST/REPLY) and processing of its effects.

This makes ordering explicit and supports controlled fault injection.

3.6 Fault Injection Design

Faults are designed to be clear and explainable:

- **Token loss:** disables progress until token regeneration.
- **Crash/recover:** marks a process as unavailable and later restored.
- **Message drops (RA):** either drop the next in-flight message (queue head) or drop-next-send (drops the next outgoing message).

These faults support demonstrating safety vs liveness trade-offs [13].

3.7 Evidence and Traceability Design

To support dissertation-quality evaluation, the tool exports an evidence triplet:

- state JSON (machine-readable),
- trace TXT (human-readable, step-numbered),
- preview PNG (visual snapshot).

The report uses curated stable figure names, linked back to raw evidence cases through a figure index. This supports reproducibility practices [20, 31, 30].

3.8 UI Design Rationale

The UI groups controls by task and exposes system status prominently:

- process table for per-process actions and states,
- message queue table for RA visibility,
- trace panel for narrative explanation,
- safety indicator for invariant status,
- preview canvas for a compact global view.

This aims to reduce extraneous cognitive load and support guided exploration [24, 15].

Chapter 4

Specification and Design

4.1 Requirements Approach

A lightweight specification was derived using a structured requirements mindset consistent with software requirements guidance [9]. The focus is on clear functional behaviour, non-functional constraints, and evidence-driven evaluation.

4.2 Functional Requirements

The core functional requirements are:

- **FR1: Configuration.** Users can select an algorithm (Token Ring or RA) and configure the number of processes.
- **FR2: Interactive execution.** Users can issue requests, release the critical section, and progress the system via Step/Run/Pause/Reset.
- **FR3: Visual status.** The UI must clearly show process states, the critical section owner, and (for RA) messages in flight.
- **FR4: Fault injection and recovery.** Users can inject representative faults and observe effects; recoveries must be supported where appropriate.
- **FR5: Scripted demos.** Repeatable scenarios can be loaded for teaching and evidence capture.
- **FR6: Evidence export.** The system exports a state snapshot and a trace, plus a visual PNG preview, enabling reproducible evidence.

4.3 Non-functional Requirements

The main non-functional requirements are:

- **NFR1: Safety.** The mutual exclusion safety property must be preserved.
- **NFR2: Clarity.** Learners must be able to explain why progress occurs or stalls (trace explanations, explicit stall reasons).

- **NFR3: Reproducibility.** Scripted scenarios and exports must allow re-running and verifying results.
- **NFR4: Portability.** The tool must run in a modern browser without external dependencies.
- **NFR5: Maintainability.** The codebase should remain small and modular to enable iteration and testing.

4.4 High-level Architecture

The design follows separation of concerns [7]:

- **Core simulation module:** maintains model state and defines algorithm step semantics.
- **UI controller:** renders tables, trace, and the preview canvas; wires user actions to core operations.

This enables headless automated tests against the core logic while keeping the UI implementation simple.

4.5 Execution Model: Step Semantics

A key design decision is discrete stepping. Each Step performs one unit of progress:

- **Token Ring:** token passing or CS entry (if the holder is requesting).
- **RA:** delivery of exactly one queued message (REQUEST/REPLY) and processing of its effects.

This makes ordering explicit and supports controlled fault injection.

4.6 Fault Injection Design

Faults are designed to be clear and explainable:

- **Token loss:** disables progress until token regeneration.
- **Crash/recover:** marks a process as unavailable and later restored.
- **Message drops (RA):** either drop the next in-flight message (queue head) or drop-next-send (drops the next outgoing message).

These faults support demonstrating safety vs liveness trade-offs [13].

4.7 Evidence and Traceability Design

To support dissertation-quality evaluation, the tool exports an evidence triplet:

- state JSON (machine-readable),
- trace TXT (human-readable, step-numbered),
- preview PNG (visual snapshot).

The report uses curated stable figure names, linked back to raw evidence cases through a figure index. This supports reproducibility practices [20, 31, 30].

4.8 UI Design Rationale

The UI groups controls by task and exposes system status prominently:

- process table for per-process actions and states,
- message queue table for RA visibility,
- trace panel for narrative explanation,
- safety indicator for invariant status,
- preview canvas for a compact global view.

This aims to reduce extraneous cognitive load and support guided exploration [24, 15].

Chapter 5

Testing

5.1 Testing Objectives

Testing provides confidence that the implementation: (i) preserves the mutual exclusion safety invariant, (ii) exhibits expected progress under normal conditions, (iii) demonstrates intended fault effects and recovery behaviours, and (iv) remains reproducible for evidence capture.

5.2 Manual Evidence-driven Testing

A structured manual test plan defines step-by-step cases for both algorithms. Each test case produces a reproducible evidence triplet (state JSON, trace TXT, preview PNG). The minimum executed set covers:

- Token Ring: basic mutual exclusion, token loss and regeneration, crash and recover.
- RA: basic request/entry/release, timestamp tie-break, message drops leading to expected stalls.

This evidence-first approach is suitable for dissertation reporting and traceable claims.

5.3 Automated Smoke Tests

Dependency-free smoke tests are implemented in Node.js and executed via `npm test`. The tests target the core simulation logic rather than UI rendering. Covered properties include:

- Token Ring safety and recovery behaviour.
- RA entry conditions and tie-break correctness.
- Expected stalls under message loss (liveness failure) without violating safety.

5.4 Continuous Integration

A GitHub Actions workflow runs smoke tests on each push and pull request, providing regression protection and an auditable record of test outcomes [3].

5.5 Limitations

Browser-level UI automation is not included. Manual tests are used to validate UI behaviour and clarity. RA liveness under message loss is intentionally not repaired via retransmission, as the project aims to demonstrate safety vs liveness trade-offs.

Chapter 6

Evaluation

6.1 Evaluation Criteria

Evaluation focuses on:

- **Correctness (Safety):** mutual exclusion is never violated.
- **Fault Demonstrations:** faults lead to explainable effects, including liveness stalls where expected.
- **Usability and Clarity:** learners can understand state, actions, and reasons for progress/stalls.
- **Reproducibility:** evidence exports and CI support traceable, repeatable results.

6.2 Correctness: Safety Invariant

Across both Token Ring and RA scenarios, the key safety invariant is preserved: at most one process is in the critical section at any time [13, 1]. The Safety indicator provides continuous visibility of this invariant during interactive exploration.

6.3 Fault Scenarios and Teaching Value

6.3.1 Token Loss and Recovery

Token loss halts progress in Token Ring until regeneration occurs. This provides a concrete demonstration that liveness can fail even when safety remains intact.

6.3.2 Crash and Recover

Crash and recover controls demonstrate how process availability influences progress. Recovery restores participation in the simulation.

6.3.3 RA Tie-break under Equal Timestamps

A scripted tie-break demo demonstrates deterministic ordering when requests share the same logical timestamp [12, 19]. PID-based tie-breaking ensures the execution remains reproducible.

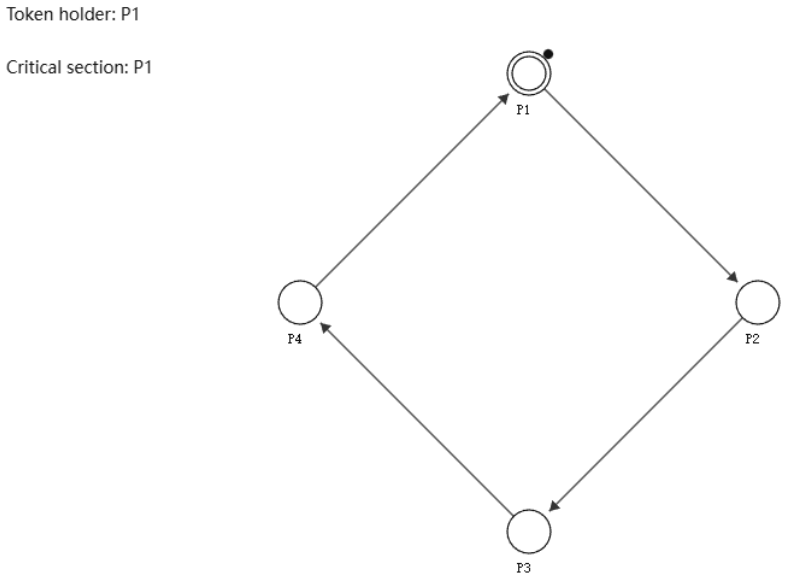

Figure 6.1: Token Ring: token loss halts progress until regeneration.

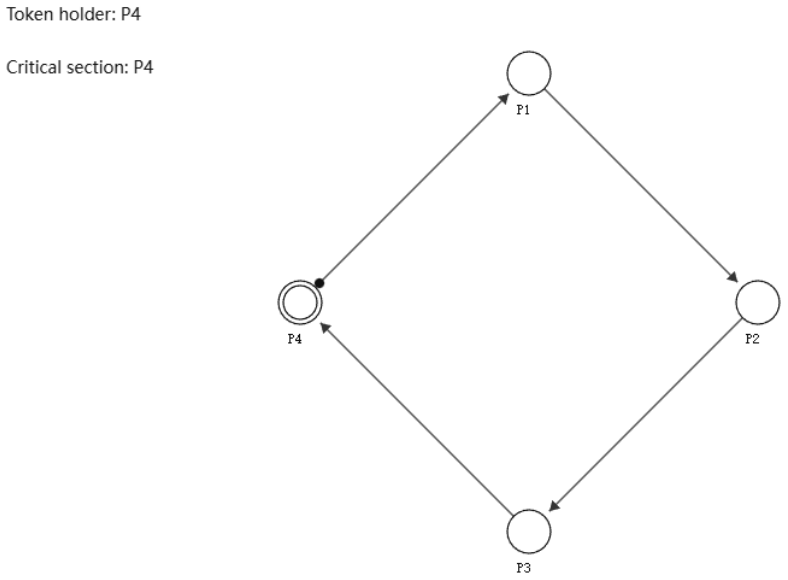

Figure 6.2: Token Ring: crash and recover demonstration.

Figure 6.3: RA: deterministic tie-break (same timestamp) using numeric PID ordering.

6.3.4 RA Message Loss: Safety vs Liveness

Message loss is injected via two mechanisms: dropping the next in-flight message and drop-next-send. Under message loss, RA may stall waiting for a missing REPLY. This is a liveness failure by design, while safety remains preserved [13].

6.4 Usability Evaluation

Usability was assessed using Nielsen heuristics [17]. Strengths include strong visibility of system status (trace, safety indicator, queue table) and user control (reset, clear trace, recover/regenerate). Improvement opportunities include adding a concise legend for preview symbols and short inline help describing what to observe (safety vs liveness). A future extension could incorporate a lightweight questionnaire such as SUS [2].

6.5 Reproducibility

Reproducibility is supported through:

- a freeze baseline (tagged release),
- CI-backed smoke tests in GitHub Actions,
- exported evidence triplets (JSON/trace/PNG),
- and indices linking report figures to raw evidence.

This workflow aligns with recommended reproducibility practices [20, 31, 30].

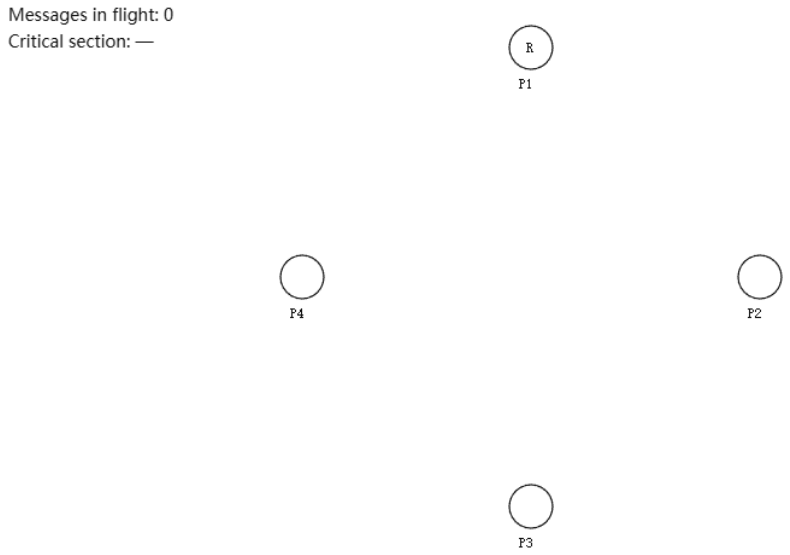

Figure 6.4: RA: dropping an in-flight message can cause a stall; the trace explains which REPLY is missing.

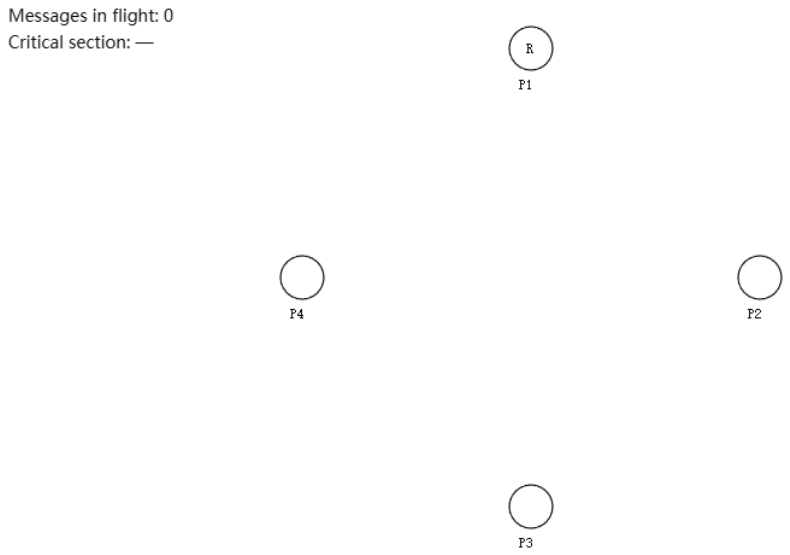

Figure 6.5: RA: drop-next-send can cause a stall while safety remains OK.

6.6 Threats to Validity

The queue-based network model is a teaching abstraction and does not represent real asynchronous timing distributions. RA is positioned as a teaching-oriented prototype; retransmission and failure detectors are not implemented. Usability evaluation is heuristic rather than a full user study; claims are therefore scoped to the employed methodology.

Chapter 7

Professional Issues

7.1 Accessibility

Accessibility is important for educational software. The interface uses clear status text (process states, Safety indicator, trace messages) and avoids relying on colour alone for critical information. Further improvements can align with WCAG and ARIA guidance, including stronger keyboard navigation, focus management, and clearer labels for interactive controls [29, 28].

7.2 Privacy and Data Protection

The tool is designed to run locally or as a static site and does not require user accounts. No personal data are collected by default. Imported and exported JSON files contain only simulation state and are stored locally by the user. This reduces privacy risk and simplifies compliance considerations.

7.3 Sustainability and Maintainability

The implementation is dependency-free and runs in a modern browser without a build step. This improves long-term sustainability by reducing supply chain and version drift risks. Version control practices and tagged releases support maintenance, traceability, and reproducible baselines [3].

7.4 Reproducibility and Research Integrity

Reproducibility is supported by a freeze baseline, CI smoke tests, and evidence exports linked to report figures via indices. This encourages verifiable claims and helps avoid overstating guarantees. The report cites all external sources, consistent with academic integrity requirements. Reproducibility practices follow published guidance [20, 31, 30].

7.5 Ethics and Responsible Communication

The tool demonstrates that faults can break liveness without violating safety, and this distinction is explicitly communicated to avoid misconceptions. RA is described as a teaching-oriented

prototype rather than a fully fault-tolerant production implementation. Limitations and assumptions are documented to support responsible interpretation of results.

7.6 Standards and Evaluation Methodology

Usability evaluation follows a heuristic approach [17] and usability concepts can be framed using ISO definitions [10]. This supports a systematic discussion of clarity, control, and status visibility.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This project delivered a browser-based Distributed Mutual Exclusion Explorer aligned with Variant 3. The tool supports interactive and scripted exploration of Token Ring and a teaching-oriented Ricart–Agrawala prototype. Fault injection (token loss, crash/recover, and message drops) enables learners to observe how assumptions affect progress. Correctness was evaluated primarily through the safety invariant of mutual exclusion, while fault scenarios were used to demonstrate the separation between safety and liveness. The project also provides a reproducible evidence pipeline and CI-backed smoke tests to support traceable reporting.

8.2 Future Work

Future work can extend both fidelity and educational value:

- Implement optional recovery strategies for RA (timeouts, retransmission, failure detection) to contrast fault tolerance approaches [4].
- Add additional mutual exclusion algorithms (e.g., Suzuki–Kasami, Maekawa, Raymond) to broaden comparisons [23, 14, 18].
- Improve the UI with a compact legend, keyboard shortcuts, and more explicit guidance on learning objectives.
- Conduct a small user study and collect structured usability metrics (e.g., SUS) to strengthen empirical evaluation [2].

Bibliography

[1] Hagit Attiya and Jennifer Welch. *Distributed Computing: Fundamentals, Simulations, and Advanced Topics.* Wiley, 2 edition, 2004.

[2] John Brooke. Sus: A "quick and dirty" usability scale. In Patrick W. Jordan, Bruce Thomas, Bernard Weerdmeester, and Ian McClelland, editors, *Usability Evaluation in Industry.* Taylor & Francis, 1996.

[3] Scott Chacon and Ben Straub. *Pro Git.* Apress, 2 edition, 2014.

[4] Tushar D. Chandra and Sam Toueg. Unreliable failure detectors for reliable distributed systems. *Journal of the ACM*, 43(2), 1996.

[5] George Coulouris, Jean Dollimore, Tim Kindberg, and Gordon Blair. *Distributed Systems: Concepts and Design.* Addison-Wesley, 5 edition, 2011.

[6] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2), 1985.

[7] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley, 1994.

[8] Christopher D. Hundhausen, Sarah A. Douglas, and John T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages and Computing*, 2002.

[9] IEEE. Ieee std 830-1998: Ieee recommended practice for software requirements specifications. IEEE Standard, 1998.

[10] International Organization for Standardization. Iso 9241-11: Ergonomics of human-system interaction — part 11: Usability: Definitions and concepts. International Standard, 2018.

[11] Paul A. Kirschner, John Sweller, and Richard E. Clark. Why minimal guidance during instruction does not work: An analysis of the failure of constructivist, discovery, problem-based, experiential, and inquiry-based teaching. *Educational Psychologist*, 41(2):75–86, 2006.

[12] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[13] Nancy A. Lynch. *Distributed Algorithms.* Morgan Kaufmann, 1996.

[14] Mamoru Maekawa. A $\sqrt{N}$ algorithm for mutual exclusion in decentralized systems. *ACM Transactions on Computer Systems*, 3(2):145–159, 1985.

[15] Richard E. Mayer. *Multimedia Learning.* Cambridge University Press, 2 edition, 2009.

[16] Thomas Naps, Guido Rößling, Vicki Almstrum, Wanda Dann, Rudolf Fleischer, Christopher Hundhausen, Ari Korhonen, Lauri Malmi, Myles McNally, Susan Rodger, and J. Á. Velázquez-Iturbide. Exploring the role of visualization and engagement in computer science education. In *Working Group Reports from ITiCSE*, 2002.

[17] Jakob Nielsen. *Usability Engineering.* Morgan Kaufmann, 1994.

[18] Kerry Raymond. A tree-based algorithm for distributed mutual exclusion. *ACM Transactions on Computer Systems*, 7(1), 1989.

[19] Glenn Ricart and Ashok K. Agrawala. An optimal algorithm for mutual exclusion in computer networks. *Communications of the ACM*, 24(1):9–17, 1981.

[20] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10), 2013.

[21] Clifford A. Shaffer, Matthew L. Cooper, Aaron J. Alon, Monika Akbar, Michael Stewart, Sean Ponce, and Stephen H. Edwards. Algorithm visualization: The state of the field. *ACM Transactions on Computing Education*, 2010.

[22] Juha Sorva, Ville Karavirta, and Lauri Malmi. A review of generic program visualization systems for introductory programming education. *ACM Transactions on Computing Education*, 2013.

[23] Ichiro Suzuki and Tadao Kasami. A distributed mutual exclusion algorithm. *ACM Transactions on Computer Systems*, 3(4):344–349, 1985.

[24] John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.

[25] Andrew S. Tanenbaum and Maarten van Steen. *Distributed Systems.* Pearson, 3 edition, 2017.

[26] Gerard Tel. *Introduction to Distributed Algorithms.* Cambridge University Press, 2 edition, 2000.

[27] Barbara Tversky, Julie Bauer Morrison, and Mireille Betrancourt. Animation: Can it facilitate? *International Journal of Human-Computer Studies*, 2002.

[28] W3C. Wai-aria 1.2. W3C Recommendation, 2021.

[29] W3C. Web content accessibility guidelines (wcag) 2.2. W3C Recommendation, 2023.

[30] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, et al. The fair guiding principles for scientific data management and stewardship. *Scientific Data*, 3, 2016.

[31] Greg Wilson, D. A. Aruliah, C. T. Brown, Neil P. C. Hong, Matt Davis, Richard T. Guy, Steven H. D. Haddock, Katy D. Huff, Ian M. Mitchell, Mark D. Plumbley, Ben Waugh, Ethan P. White, and Paul Wilson. Best practices for scientific computing. *PLoS Biology*, 12(1), 2014.

Appendix A

User Manual

Run a local server from the repository root:

```
python -m http.server 5500
```

Open `http://localhost:5500/`. Use Step/Run to progress, inject faults, and export evidence.

Appendix B

Test Cases

This appendix summarises the manual test cases executed for v0.3.2, including Token Ring (TR-01–TR-03) and RA (RA-01–RA-04).